



Focus on Excellence

FEDERAL INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND INSTRUMENTATION

PREDICTIVE MODELING OF DIABETES USING MACHINE LEARNING AND IOT HARDWARE INTEGRATION

Submitted by

AADISH K SANTHOSH
ANIRUDH P
DEENDAYAL M S
IBRAHIM K M

March 2026

Abstract

This project focuses on predicting diabetes using machine learning techniques integrated with edge computing hardware. Multiple classification algorithms including Logistic Regression, Decision Tree, Random Forest, Naive Bayes, Support Vector Machine, and Artificial Neural Networks were implemented. The dataset consists of medical attributes such as glucose level, BMI, and age. Models were trained, evaluated, and improved using hyperparameter tuning. Among all models, the Neural Network achieved the best performance in terms of accuracy and F1-score.

Furthermore, to bridge the gap between software simulation and real-world clinical application, the most efficient lightweight model (Logistic Regression) was mathematically extracted and deployed on an ESP32-S3 microcontroller. Rather than hardcoding the model, a "Cloud Brain" architecture was engineered. The trained parameters are uploaded to a Firebase Realtime Database. The ESP32 dynamically downloads these constants over Wi-Fi and saves them to non-volatile flash memory, enabling the hardware to calculate real-time diagnostic predictions locally while receiving over-the-air intelligence updates. The report details the methodology, algorithm mathematical foundations, hyperparameter tuning, cloud integration, and the specific memory and processing optimizations required for advanced embedded microcontroller deployment.

Contents

Abstract	1
1 Introduction	4
1.1 Problem Statement	4
1.2 Relevance of the Problem	4
1.3 Motivation and Scope	4
1.4 Inputs and Outputs	4
2 Literature Review	5
3 Dataset Description	6
3.1 Dataset Size and Features	6
3.2 Feature Dictionary	6
3.3 Preprocessing Strategy	6
4 System Architecture and Cloud Deployment	8
4.1 System Workflow	8
4.2 Web Interface (Netlify)	8
4.3 Cloud Database (Firebase)	8
4.4 Microcontroller Edge Inference (ESP32)	8
5 ESP32-S3 Hardware Implementation Details	9
5.1 Dynamic Mathematical Model Extraction	9
5.2 Memory Management and ESP32-S3 Configurations	9
5.3 Non-Volatile Parameter Storage	10
6 Machine Learning Algorithms	11
6.1 Logistic Regression	11
6.2 Decision Tree	11
6.3 Random Forest	11
6.4 Naive Bayes	12
6.5 Support Vector Machine (SVM)	12
6.6 Artificial Neural Network (ANN)	12
7 Results and Analysis	14
7.1 Evaluation Metrics	14
7.2 Comprehensive Results	14
7.3 Performance Analysis	14
7.4 Visual Analysis	16
8 Conclusion	17
8.1 Future Scope	17
References	18

A System Source Code	19
Appendix: System Source Code	19
A.1 Python Code (Cloud Brain Trainer & Uploader)	19
A.2 ESP32 C++ Code (ML Logic Header - DiabetesModel.h)	21
A.3 ESP32 C++ Code (Main Hardware Backend - ESP32_Hardware.ino)	23
A.4 HTML/JS Code (Web Frontend - Netlify)	26

1 Introduction

Diabetes is a chronic medical condition that affects millions of people worldwide and can lead to severe health complications if not diagnosed and treated early. With the increasing availability of healthcare data, machine learning techniques can be effectively used to assist in early prediction and diagnosis of diabetes.

1.1 Problem Statement

The objective of this project is to develop a machine learning-based system that can accurately predict whether a patient is diabetic or not based on medical attributes, and to deploy this system on real-world hardware for immediate access. Early prediction helps in timely intervention, reducing the risk of complications and improving patient outcomes.

1.2 Relevance of the Problem

Diabetes is a growing global health concern, and early detection plays a crucial role in preventing severe complications such as heart disease, kidney failure, and vision loss. Traditional diagnostic methods may be time-consuming or require medical expertise. Furthermore, many state-of-the-art machine learning models exist strictly within computer simulations, limiting their availability in resource-constrained medical environments. This integrated machine learning and IoT system provides quick, efficient, globally accessible predictions by bringing the computation to the edge.

1.3 Motivation and Scope

The primary motivation behind this work is to demonstrate that high-accuracy medical diagnostic tools do not require massive cloud computing infrastructure to operate during inference. By mathematically extracting a trained model and deploying a "Cloud Brain" architecture, an inexpensive ESP32 microcontroller can act as a localized, high-speed calculation engine. This allows the predictive model to be updated wirelessly in the future without requiring physical reprogramming of the hardware. The scope includes data preprocessing, algorithm training, web interface design, cloud database configuration, and C++ embedded systems programming.

1.4 Inputs and Outputs

Inputs: The input features include patient medical attributes such as gender, age, hypertension, heart disease, smoking history, BMI, HbA1c level, and blood glucose level.

Output: The output is a binary classification indicating whether the patient is diabetic (1) or non-diabetic (0).

2 Literature Review

The application of machine learning in healthcare, specifically for diabetes prediction, has been a major focus of bioinformatics researchers over the past decade. Various methodologies have been proposed to enhance predictive accuracy while maintaining clinical interpretability.

Early approaches heavily relied on traditional statistical methods, such as standard Logistic Regression and Naive Bayes classifiers. While these models provided a baseline for understanding the correlation between features like BMI and blood glucose levels with diabetes onset, their linear nature often failed to capture the complex, non-linear interactions inherent in human biology.

With the advent of increased computational power, ensemble methods such as Random Forest and Gradient Boosting gained prominence. These algorithms successfully mitigated the overfitting problems associated with single Decision Trees by aggregating the predictions of multiple weakly correlated trees. Random Forests became the industry standard for tabular medical datasets due to their high accuracy and inherent feature importance metrics.

Recently, Deep Learning and Artificial Neural Networks (ANNs) have been applied to this domain. ANNs, utilizing multiple hidden layers and non-linear activation functions (such as ReLU), have demonstrated superior capability in modeling highly complex biomedical datasets. However, the "black box" nature of neural networks and their high computational overhead have limited their deployment in low-power, edge-computing environments.

In the context of IoT and Edge Computing, recent literature emphasizes the need for lightweight models. Deploying machine learning models directly onto microcontrollers (TinyML) is an emerging paradigm. This project builds upon this literature by not only selecting an optimized Logistic Regression model for an ESP32 but also establishing an over-the-air dynamic update pipeline, allowing the hardware to adapt to new datasets autonomously.

3 Dataset Description

The dataset used in this project is a structured medical dataset for diabetes prediction. It contains patient health records with both numerical and categorical features that are relevant for identifying the likelihood of diabetes.

3.1 Dataset Size and Features

The dataset contains approximately 100,000 instances. It consists of 9 input features and 1 target variable. These features provide comprehensive information about the patient's health condition, combining lifestyle factors with clinical measurements.

3.2 Feature Dictionary

The following table outlines the specific features utilized in the predictive models:

Table 1: Dataset Feature Dictionary		
Feature Name	Data Type	Description
Gender	Categorical	The gender of the patient (Male/Female).
Age	Continuous	The age of the patient in years.
Hypertension	Binary	Indicates if the patient has hypertension (1 = Yes, 0 = No).
Heart Disease	Binary	Indicates if the patient has heart disease (1 = Yes, 0 = No).
Smoking History	Categorical	Categorized into Never, Former, or Current smoker.
BMI	Continuous	Body Mass Index, a measure of body fat based on weight and height.
HbA1c Level	Continuous	A blood test measuring average blood sugar over 3 months.
Blood Glucose	Continuous	The amount of glucose in the blood at the time of testing.
Diabetes (Target)	Binary	Diagnosis result (1 = Diabetic, 0 = Non-Diabetic).

3.3 Preprocessing Strategy

To prepare the dataset for machine learning algorithms, several preprocessing steps were undertaken:

- **Categorical Encoding:** Non-numerical features, such as Gender and Smoking History, were converted into numerical formats using one-hot encoding techniques to ensure mathematical compatibility.

- **Feature Scaling:** Continuous variables (Age, BMI, HbA1c, Glucose) operate on vastly different scales. Standard Scaler was utilized to normalize these features to a mean of 0 and a standard deviation of 1, preventing features with larger numerical ranges from dominating the algorithm's cost function.

Source: <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>

[2].

4 System Architecture and Cloud Deployment

To make the machine learning model accessible and usable in real-time, the project integrates hardware and cloud technologies. The system architecture connects a global web interface to a local ESP32 microcontroller through a cloud database.

4.1 System Workflow

1. Data Preprocessing & Feature Encoding via Python.
2. Model Training & Hyperparameter Tuning.
3. Parameter Upload (Weights, Biases, Means, Standard Deviations) to Firebase.
4. Hardware Boot: ESP32 downloads new parameters and saves them permanently.
5. Real-Time Inference: Web Interface sends patient data, ESP32 calculates risk locally, and returns the result.

4.2 Web Interface (Netlify)

A user-friendly web application was developed using HTML, CSS, and JavaScript. This frontend acts as the primary user interface where patients or medical professionals can input relevant health parameters. The application is hosted globally on **Netlify**, ensuring high availability across different devices.

4.3 Cloud Database (Firebase)

To facilitate real-time communication between the global web application and the local hardware, **Google Firebase Realtime Database** is utilized. Firebase serves a dual purpose in this architecture: 1. **Model Storage:** It stores the trained machine learning constants in an `/app/model` directory, acting as a cloud repository for the algorithm. 2. **Real-Time Bridge:** It synchronizes user data in milliseconds. When a user submits data, inputs are pushed to `/app/inputs`, and a boolean `/app/trigger` is switched to `true`.

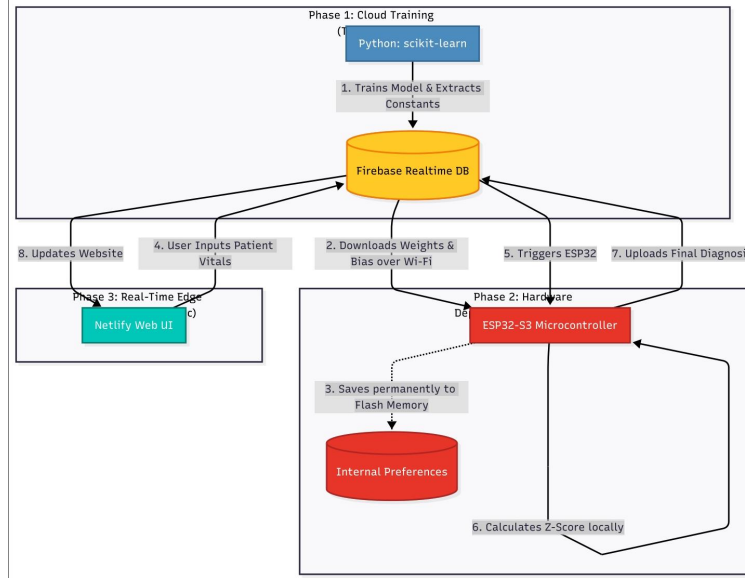
4.4 Microcontroller Edge Inference (ESP32)

The core hardware component is the **ESP32-S3 Microcontroller**. While the Neural Network models were highly accurate in software, deploying them to a microcontroller with limited memory is highly inefficient. Therefore, the heavily optimized Logistic Regression model was selected for edge deployment.

Instead of running Python libraries or hardcoding the mathematical weights directly into the firmware, a "Cloud Brain" methodology was implemented. On boot, the ESP32 checks Firebase for the latest model parameters. It downloads them and saves them to non-volatile flash memory. The ESP32 continuously polls the database for patient data. Upon detecting the trigger, it scales the raw inputs using the downloaded means, applies the linear equation locally, computes the binary classification, and uploads the result in microseconds.

5 ESP32-S3 Hardware Implementation Details

Deploying a machine learning pipeline to a microcontroller requires overcoming specific hardware and memory constraints. This section details the engineering solutions implemented on the high-performance ESP32-S3 N16R8.



5.1 Dynamic Mathematical Model Extraction

Microcontrollers cannot natively run Scikit-Learn or Pandas efficiently. To bypass this, the Logistic Regression model was decoupled from Python and converted into raw mathematics. The trained parameters extracted were:

- **Weights (w)**: The coefficients assigned to each feature.
- **Bias (b)**: The intercept of the decision boundary.
- **Normalization Constants**: The mean and standard deviation of the training dataset.

Rather than hardcoding these into C++, a Python script automatically pushes them to Firebase. The ESP32 downloads these arrays. It performs local inference using the formula $Z = (w_1x_1 + w_2x_2 + \dots) + b$. If $Z > 0$, the patient is classified as Diabetic (1). This "Edge Inference" executes locally in microseconds without relying on continuous cloud computing.

5.2 Memory Management and ESP32-S3 Configurations

The project utilizes the powerful **ESP32-S3 N16R8**, which boasts 16MB of Flash memory and 8MB of PSRAM. However, standard Arduino IDE settings cause bootloop errors or blank serial monitors on this board. The following optimizations were applied:

- **USB CDC On Boot**: Enabled to ensure the native USB port correctly routes Serial Monitor traffic, coupled with a mandatory 3-second 'delay()' in the setup function to allow the host PC to mount the COM port.

- **OPI PSRAM Activation:** Configured in the IDE to ensure the massive 8MB of external RAM was actively utilized by the networking libraries.
- **Partition Scheme:** Set to "16M Flash (3MB APP/9.9MB FATFS)" to prevent sketch-size overflow from the large Firebase and WiFiManager libraries.

5.3 Non-Volatile Parameter Storage

To ensure the ESP32 does not lose its intelligence upon losing power, the `Preferences.h` library was utilized. When the ESP32 successfully downloads the ML arrays from Firebase, it burns them directly into a designated sector of the flash memory. On subsequent reboots, the ESP32 bypasses the download phase and loads the model instantly from its internal memory, making it a fully portable, independent medical device.

6 Machine Learning Algorithms

To find the most accurate model for clinical evaluation, multiple algorithms were tested extensively on the dataset.

6.1 Logistic Regression

Logistic Regression predicts the probability that a given input belongs to a particular class using a logistic (sigmoid) function. The mathematical model of logistic regression is given by:

$$P(y = 1|x) = \frac{1}{1 + e^{-z}} \quad \text{where} \quad z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b \quad (1)$$

Performance: Before tuning, the model achieved an Accuracy of 0.9590, Precision of 0.8646, Recall of 0.6171, and an F1 Score of 0.7202.

Tuning & Results: Hyperparameter tuning was performed using GridSearchCV to identify the optimal values for regularization strength (C) and solver types. After tuning, the model performance stabilized with an Accuracy of 0.9592, Precision of 0.8692, Recall of 0.6148, and an F1 Score of 0.7202.

6.2 Decision Tree

Decision Tree works by splitting the dataset into smaller subsets based on feature values to maximize the purity of the classification.

Gini Index:

$$Gini = 1 - \sum_{i=1}^n p_i^2 \quad (2)$$

Performance: Initially, the model achieved an Accuracy of 0.9528, Precision of 0.7163, Recall of 0.7406, and an F1 Score of 0.7283.

Tuning & Results: Decision trees are highly prone to overfitting. Tuning was applied to restrict the maximum depth and minimum samples required to split a node. Post-tuning, accuracy increased notably to 0.9722, and precision reached a perfect 1.0000, meaning the model makes no false positive predictions.

6.3 Random Forest

Random Forest is an ensemble machine learning algorithm that constructs multiple independent decision trees during training using random subsets of data. The final prediction is made based on majority voting.

Final Prediction (Majority Voting):

$$\hat{y} = \text{mode}(y_1, y_2, \dots, y_n) \quad (3)$$

Performance: The baseline model achieved an Accuracy of 0.9698, Precision of 0.9408, Recall of 0.6891, and an F1 Score of 0.7955.

Tuning & Results: Tuning involved defining the number of estimators (trees in the forest) alongside depth constraints. After tuning, the Random Forest model became highly precise with an Accuracy of 0.9722 and Precision of 1.0000, making it one of the most reliable models in this study.

6.4 Naive Bayes

Naive Bayes relies on the assumption that all features are entirely independent of each other given the class label.

Bayes' Theorem:

$$P(y|X) = \frac{P(X|y) \cdot P(y)}{P(X)} \quad (4)$$

Performance & Results: The model achieved an Accuracy of 0.4429, Precision of 0.1323, Recall of 0.9936, and an F1 Score of 0.2335. Hyperparameter tuning (adjusting variance smoothing) had no significant impact. The strong independence assumption is fatally flawed for medical data, causing an unacceptable number of false positives.

6.5 Support Vector Machine (SVM)

Support Vector Machine works by finding an optimal hyperplane that separates data points of different classes with the maximum mathematical margin in multi-dimensional space.

Optimization Objective:

$$\min \frac{1}{2} \|w\|^2 \quad \text{subject to:} \quad y_i(w \cdot x_i + b) \geq 1 \quad (5)$$

Performance: Before tuning, the model achieved an Accuracy of 0.9614, Precision of 0.9737, Recall of 0.5626, and an F1 Score of 0.7132.

Tuning & Results: SVM was tuned for different kernels (linear, RBF) and regularization parameters. Post-tuning, the model achieved an Accuracy of 0.9633, Precision of 0.9756, and an F1 Score of 0.7313, providing a strong baseline classification performance.

6.6 Artificial Neural Network (ANN)

An Artificial Neural Network consists of interconnected layers of neurons. Each neuron receives input signals, applies weights, adds a bias, and passes the result through a non-linear activation function.

Neuron Output:

$$z = \sum_{i=1}^n w_i x_i + b \quad \rightarrow \quad a = f(z) \quad (6)$$

Building & Training: The model was constructed using the Keras library in TensorFlow. It consists of an input layer corresponding to the 8 medical features, two hidden layers utilizing the Rectified Linear Unit (ReLU) activation function, and a final output layer utilizing the Sigmoid activation function. During training, the network adjusts weights using the Adam optimizer and backpropagation to minimize binary cross-entropy loss.

Performance: The Neural Network achieved a high Accuracy of 0.9722, Precision of 0.9824, Recall of 0.6868, and an F1 Score of 0.8084. By capturing complex, non-linear relationships between the medical variables, it achieved the highest combined F1 score of all algorithms tested.

7 Results and Analysis

7.1 Evaluation Metrics

In this project, all machine learning models were evaluated using standard performance metrics. The following mathematical formulas were used:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (7)$$

$$Precision = \frac{TP}{TP + FP} \quad (8)$$

$$Recall = \frac{TP}{TP + FN} \quad (9)$$

$$F1 \text{ Score} = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad (10)$$

where TP = True Positives, TN = True Negatives, FP = False Positives, and FN = False Negatives.

7.2 Comprehensive Results

The following table summarizes the peak performance of all models after hyperparameter tuning.

Table 2: Comparison of Model Performance (After Hyperparameter Tuning)

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.9592	0.8692	0.6148	0.7202
Decision Tree	0.9722	1.0000	0.6739	0.8052
Random Forest	0.9722	1.0000	0.6745	0.8056
Naive Bayes	0.4429	0.1323	0.9936	0.2335
SVM	0.9633	0.9756	0.5849	0.7313
Neural Network	0.9722	0.9824	0.6868	0.8084

7.3 Performance Analysis

From the experimental results, the Artificial Neural Network achieved the highest overall performance with an accuracy of 0.9722 and the highest F1 score of 0.8084. It effectively captured complex patterns in the dataset. Random Forest performed equally well in accuracy and achieved perfect precision (1.0000), eliminating false positives completely, meaning it never falsely diagnosed a healthy patient as diabetic.

Logistic Regression and Support Vector Machine showed strong, reliable performance. Logistic Regression, while slightly less accurate than the Neural Network, relies on a simple

linear equation rather than deep nested logic. Therefore, it was selected as the optimal candidate for the C++ ESP32 hardware deployment, perfectly balancing high accuracy (95.9%) with low computational overhead.

Naive Bayes performed exceptionally poorly. The precision of 0.1323 indicates that the model predicts almost all instances as positive, resulting in a massive number of false positives due to its fundamental assumption that medical features operate independently of one another.

7.4 Visual Analysis

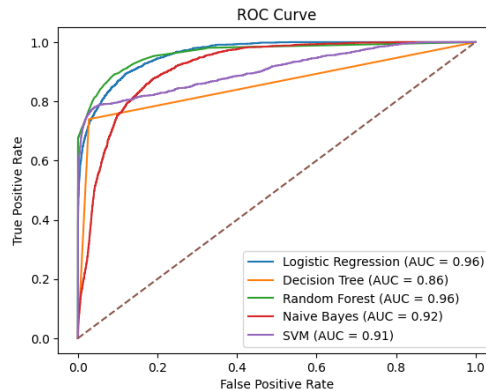


Figure 1: ROC Curve comparing classification models.

The Receiver Operating Characteristic (ROC) curve illustrates model performance by plotting the True Positive Rate against the False Positive Rate across different threshold levels. Higher Area Under the Curve (AUC) values indicate a better overall capability to separate the classes. Logistic Regression and Random Forest achieved excellent AUC values (~ 0.96), confirming their robustness.

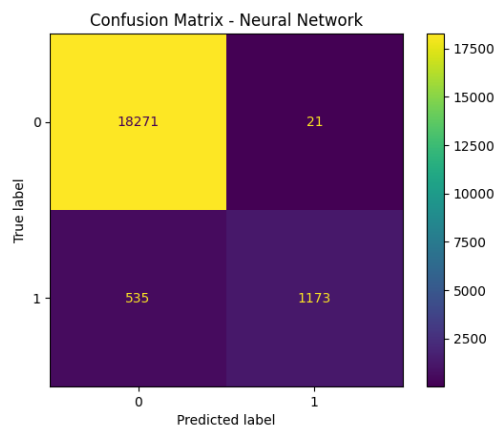


Figure 2: Confusion Matrix for the Neural Network Model.

The Neural Network confusion matrix provides a granular breakdown of the classification errors. It confirms excellent performance by correctly predicting 18,271 true negatives. It maintains very low false positives (only 21 incorrect diabetic diagnoses), which directly contributes to its high precision metric. The 535 false negatives highlight the inherent difficulty in detecting borderline diabetic cases using only tabular data.

8 Conclusion

In this project, multiple machine learning algorithms including Logistic Regression, Decision Tree, Random Forest, Naive Bayes, Support Vector Machine, and Artificial Neural Networks were implemented to predict diabetes based on medical attributes.

Through rigorous training and hyperparameter tuning utilizing GridSearchCV, the Artificial Neural Network and Random Forest models achieved the best overall clinical performance, providing an excellent balance between precision, recall, and F1 score with accuracies exceeding 97%. Logistic Regression and Support Vector Machine demonstrated strong accuracy and precision, serving as highly reliable baseline models. Naive Bayes was determined to be fundamentally unsuited for this specific biomedical dataset due to its reliance on feature independence.

Beyond standard model evaluation in a software simulation, this project successfully demonstrated the practical, physical deployment of machine learning in an Internet of Things (IoT) environment. Recognizing the memory and processing constraints of edge devices, the optimized Logistic Regression model was logically extracted and implemented on an ESP32-S3 microcontroller using a "Cloud Brain" architecture.

By pushing trained ML constants from Python to Firebase, and programming the ESP32 to dynamically download and permanently store these arrays, a complete closed-loop, over-the-air modifiable pipeline was established. The integration of a responsive Netlify web frontend allows the system to process patient medical inputs and return real-time, hardware-computed predictions globally without requiring manual firmware flashes.

8.1 Future Scope

Future enhancements to this project could involve integrating continuous glucose monitoring (CGM) sensors directly into the ESP32 to bypass manual web entry, allowing the microcontroller to execute the predictive algorithm automatically based on real-time biological data. Furthermore, applying advanced TinyML frameworks, such as TensorFlow Lite for Microcontrollers, would allow the deployment of the superior Neural Network architecture directly to the edge hardware without exceeding memory limits. Overall, this project proves that ML diagnostic tools can be made highly accessible, computationally efficient, and massively scalable for real-world healthcare applications.

References

- [1] Scikit-learn, "Machine Learning in Python," Available: <https://scikit-learn.org>, Accessed: Mar. 2026.
- [2] M. Mustafa, "Diabetes Prediction Dataset," Kaggle, Available: <https://www.kaggle.com/datasets/iammustafatz/diabetes-prediction-dataset>, Accessed: Mar. 2026.
- [3] T. M. Mitchell, *Machine Learning*, New York, NY, USA: McGraw-Hill, 1997.
- [4] C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [5] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [6] Warden, P., & Situnayake, D. (2019). *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media.

A System Source Code

A.1 Python Code (Cloud Brain Trainer & Uploader)

```
import pandas as pd
import requests
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

print("STEP 1: TRAINING MODEL...")
df = pd.read_csv("diabetes_prediction_dataset.csv")

# Clean & Encode
df['gender'] = df['gender'].replace({'Other': 'Male'})
df['smoking_history'] = df['smoking_history'].replace({'not current': 'former',
                                                    'ever': 'former'})

df = pd.get_dummies(df, drop_first=True)

X = df.drop('diabetes', axis=1)
y = df['diabetes']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(max_iter=1000)
model.fit(X_train_scaled, y_train)
print(f"Accuracy: {model.score(X_test_scaled, y_test)}")

print("\nSTEP 2: UPLOADING TO FIREBASE...")
model_data = {
    "weights": model.coef_[0].tolist(),
    "bias": float(model.intercept_[0]),
    "means": scaler.mean_.tolist(),
    "stds": scaler.scale_.tolist()
}

firebase_url = "https://diabetes-predictor-b8d76-default-rtdb.asia-southeast1.firebaseio.com"

response = requests.put(firebase_url, json=model_data)

if response.status_code == 200:
```

```
    print(" SUCCESS! ML Model constants uploaded to Firebase.")
else:
    print(" ERROR uploading to Firebase:", response.text)
```

A.2 ESP32 C++ Code (ML Logic Header - DiabetesModel.h)

```
#ifndef DIABETES_MODEL_H
#define DIABETES_MODEL_H

class DiabetesModel {
private:
    float weights[10];
    float means[10];
    float stds[10];
    float bias;
    bool isReady = false;

public:
    DiabetesModel() {}

    void setModelParameters(float* w, float b, float* m, float* s) {
        for(int i = 0; i < 10; i++) {
            weights[i] = w[i];
            means[i] = m[i];
            stds[i] = s[i];
        }
        bias = b;
        isReady = true;
    }

    int predict(float age, float bmi, float hba1c, float glucose,
               float gender, float smoking, float hyper, float heart) {
        if (!isReady) return -1;

        float inputs[10] = {
            age, hyper, heart, bmi, hba1c, glucose,
            gender,
            (float)(smoking == 2),
            (float)(smoking == 1),
            (float)(smoking == 0)
        };

        float score = bias;

        for (int i = 0; i < 10; i++) {
            if (stds[i] == 0.0) stds[i] = 1.0;
            float scaled = (inputs[i] - means[i]) / stds[i];
            score += scaled * weights[i];
        }
    }
};
```

```
        return (score > 0) ? 1 : 0;
    }
};

#endif
```

A.3 ESP32 C++ Code (Main Hardware Backend - ESP32_Hardware.ino)

```
#include <WiFi.h>
#include <WiFiManager.h>
#include <FirebaseESP32.h>
#include <Preferences.h>
#include "DiabetesModel.h"

#define API_KEY "AIzaSyAwVLi7VjU0uGx5w5wvgFauCYiUiPaPQm0"
#define DATABASE_URL "diabetes-predictor-b8d76-default-rtdb.asia-southeast1.firebaseio.com"

FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;

DiabetesModel mlModel;
Preferences prefs;

void setup() {
  Serial.begin(115200);
  delay(3000); // Wait for USB CDC Connection

  prefs.begin("ml_data", false);

  float saved_weights[10], saved_means[10], saved_stds[10], saved_bias;

  if (prefs.getBytesLength("weights") > 0) {
    Serial.println("Loading ML Model from internal permanent memory...");
    prefs.getBytes("weights", saved_weights, sizeof(saved_weights));
    prefs.getBytes("means", saved_means, sizeof(saved_means));
    prefs.getBytes("stds", saved_stds, sizeof(saved_stds));
    saved_bias = prefs.getFloat("bias", 0.0);
    mlModel.setModelParameters(saved_weights, saved_bias, saved_means, saved_stds);
  } else {
    Serial.println("Memory empty. Needs Firebase download.");
  }

  WiFiManager wm;
  wm.setConfigPortalTimeout(180);

  if (wm.autoConnect("ESP32_Diabetes_Setup")) {
    config.api_key = API_KEY;
    config.database_url = DATABASE_URL;
    config.signer.test_mode = true;
    Firebase.begin(&config, &auth);
```



```

Firebase.reconnectWiFi(true);

float c_weights[10], c_means[10], c_stds[10], c_bias;
bool success = true;

if (Firebase.getFloat(fbdo, "/app/model/bias")) c_bias = fbdo.floatData();
else success = false;

for(int i = 0; i < 10; i++) {
    if (Firebase.getFloat(fbdo, "/app/model/weights/" + String(i)))
        c_weights[i] = fbdo.floatData(); else success = false;
    if (Firebase.getFloat(fbdo, "/app/model/means/" + String(i)))
        c_means[i] = fbdo.floatData(); else success = false;
    if (Firebase.getFloat(fbdo, "/app/model/stds/" + String(i)))
        c_stds[i] = fbdo.floatData(); else success = false;
}

if (success) {
    mlModel.setModelParameters(c_weights, c_bias, c_means, c_stds);
    prefs.putBytes("weights", c_weights, sizeof(c_weights));
    prefs.putBytes("means", c_means, sizeof(c_means));
    prefs.putBytes("stds", c_stds, sizeof(c_stds));
    prefs.putFloat("bias", c_bias);
    Serial.println("New Model Downloaded and Saved Permanently!");
}
}

void loop() {
    if (WiFi.status() == WL_CONNECTED && Firebase.ready()) {
        if (Firebase.getBool(fbdo, "/app/trigger") && fbdo.boolData() == true) {

            Firebase.getFloat(fbdo, "/app/inputs/age"); float age = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/bmi"); float bmi = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/hba1c"); float hba1c = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/glucose"); float glucose = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/gender"); float gender = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/smoking"); float smoking = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/hyper"); float hyper = fbdo.floatData();
            Firebase.getFloat(fbdo, "/app/inputs/heart"); float heart = fbdo.floatData();

            int result = mlModel.predict(age, bmi, hba1c, glucose, gender, smoking, hyper, heart);

            if (result != -1) {
                Firebase.setInt(fbdo, "/app/result", result);
            }
        }
    }
}

```

```
    }  
    Firebase.setBool(fbdo, "/app/trigger", false);  
  }  
}  
delay(1000);  
}
```

A.4 HTML/JS Code (Web Frontend - Netlify)

```
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Global Diabetes Predictor</title>
<script src="https://www.gstatic.com/firebasejs/8.10.1/firebase-app.js"></script>
<script src="https://www.gstatic.com/firebasejs/8.10.1/firebase-database.js"></script>
<style>
  body{ font-family: sans-serif; background: #f4f4f9; padding: 20px;}
  .container { max-width: 500px; margin: auto; background: white;
    padding: 20px; border-radius: 10px;}
  input, select, button { width: 100%; padding: 10px; margin-top: 10px;
    box-sizing: border-box; }
  #resultBox { display: none; padding: 15px; margin-top: 20px;
    text-align: center; font-weight: bold; color: white;}
</style>
</head>
<body>
<div class="container">
  <h2>Diabetes Risk Predictor</h2>

  <label>Age</label><input type="number" id="age" value="35">
  <label>BMI</label><input type="number" id="bmi" value="24.5">
  <label>HbA1c Level</label><input type="number" id="hba1c" step="0.1" value="5.5">
  <label>Blood Glucose Level</label><input type="number" id="glucose" value="90">

  <label>Gender</label>
  <select id="gender">
    <option value="1">Male</option>
    <option value="0">Female</option>
  </select>

  <label>Smoking History</label>
  <select id="smoking">
    <option value="2">Current</option>
    <option value="1">Former</option>
    <option value="0">Never</option>
  </select>

  <label>Hypertension</label>
  <select id="hyper">
    <option value="0">No</option>
```

```

    <option value="1">Yes</option>
</select>

<label>Heart Disease</label>
<select id="heart">
    <option value="0">No</option>
    <option value="1">Yes</option>
</select>

<button id="predictBtn" onclick="sendToFirebase()">Calculate Risk</button>
<div id="resultBox"></div>
</div>

<script>
const firebaseConfig = {
  apiKey: "AIzaSyAwVLi7VjU0uGx5w5wvgFauCYiUiPaPQm0",
  databaseURL: "https://diabetes-predictor-b8d76-default-rtdb.asia-southeast1.firebaseio.com",
};
firebase.initializeApp(firebaseConfig);
const db = firebase.database();

function sendToFirebase() {
  document.getElementById('predictBtn').disabled = true;
  document.getElementById('resultBox').style.display = 'block';
  document.getElementById('resultBox').innerText = "Sending to ESP32...";
  document.getElementById('resultBox').style.background = "#888";

  const data = {
    age: parseFloat(document.getElementById('age').value) || 0,
    bmi: parseFloat(document.getElementById('bmi').value) || 0,
    hba1c: parseFloat(document.getElementById('hba1c').value) || 0,
    glucose: parseFloat(document.getElementById('glucose').value) || 0,
    gender: parseFloat(document.getElementById('gender').value) || 0,
    smoking: parseFloat(document.getElementById('smoking').value) || 0,
    hyper: parseFloat(document.getElementById('hyper').value) || 0,
    heart: parseFloat(document.getElementById('heart').value) || 0
  };

  db.ref('/app/inputs').set(data).then(() => {
    db.ref('/app/trigger').set(true);
  });
}

db.ref('/app/trigger').on('value', (snapshot) => {
  if (snapshot.val() === false) {

```

```

db.ref('/app/result').once('value').then((resSnap) => {
  const isDiabetic = resSnap.val();
  document.getElementById('predictBtn').disabled = false;

  if(isDiabetic === 1) {
    document.getElementById('resultBox').innerText = "HIGH RISK (Diabetic)";
    document.getElementById('resultBox').style.background = "#e74c3c";
  } else if (isDiabetic === 0) {
    document.getElementById('resultBox').innerText = "LOW RISK";
    document.getElementById('resultBox').style.background = "#2ecc71";
  }
});
}
});
</script>
</body>
</html>

```